
Tutorial 4

Computer Security BCS2420

Part A

1. Which statement best describes the difference between *key transport* and *key agreement*?
 - ☐ In key transport, the shared key is a result of both parties' private contributions, while in key agreement one party unilaterally chooses the session key.
 - ☐ In key transport, one party picks the key and securely sends it; in key agreement, both parties' inputs determine the shared key.
 - ☐ Key transport always ensures forward secrecy, while key agreement never does.
 - ☐ Key transport is used only for symmetric algorithms; key agreement is used only for asymmetric algorithms.
2. Which of the following is not a time-variant parameter (TVP) typically used to prevent replay attacks?
 - ☐ Nonce
 - ☐ Timestamp
 - ☐ Sequence number
 - ☐ Symmetric key
3. In a Diffie-Hellman key exchange, if an active attacker performs a *middle-person* (man-in-the-middle) attack, what is the primary vulnerability being exploited?
 - ☐ The parties do not verify each other's public parameters.
 - ☐ The prime modulus (p) is too large for practical usage.
 - ☐ The discrete log problem is easy to solve.
 - ☐ Both endpoints are using ephemeral keys with perfect

forward secrecy.

4. Which statement about a *reflection attack* is **accurate**?
- ☐ It is an attack where the adversary tries many dictionary passwords offline.
 - ☐ It involves sending a copy of a message back to the original sender to confuse or bypass authentication steps.
 - ☐ It uses advanced polynomial-time algorithms to invert a one-way function.
 - ☐ It always requires physical proximity to the victim's device.
5. Which scenario best illustrates a *relay attack*?
- ☐ Reusing an old message to trick the server into accepting stale credentials
 - ☐ Actively forwarding protocol messages in real time between two unsuspecting parties to impersonate one side
 - ☐ Using reflection to replay the server's random challenge back to it
 - ☐ Flooding the network with random guesses of session keys
6. In an entity-authentication protocol, why might using a simple hash of the secret key (e.g., $H(K)$) to prove knowledge be insecure?
- ☐ Hash functions are always collision-resistant, so replays cannot occur.
 - ☐ Offline attackers can replay $H(K)$ without knowing K .
 - ☐ The hash output is too large to fit in typical protocol messages.
 - ☐ The server must reveal its own secret for the handshake to complete.
7. Which property distinguishes *implicit key authentication* from *explicit key authentication*?
- ☐ Explicit key authentication ensures the key is not only known to the correct parties but also confirmed by them, while implicit authentication only assures the key is shared by correct parties.
 - ☐ Implicit key authentication uses ephemeral keys, explicit uses static keys.
 - ☐ Implicit key authentication always provides forward

- secrecy, explicit never does.
- ☐ They are identical properties; the terms are interchangeable.
8. Why is forward secrecy often desired in key-establishment protocols?
- ☐ It allows for usage of older, weaker ciphers without risk.
 - ☐ It ensures that compromising a long-term key does not reveal *past* session keys.
 - ☐ It eliminates the need for ephemeral keys entirely.
 - ☐ It requires fewer computational resources on each side.
9. Which of the following is not a fundamental challenge in key management for secure protocols?
- ☐ Securely generating random keys
 - ☐ Preventing key reuse across multiple sessions
 - ☐ Keeping track of large prime factorization results for future reference
 - ☐ Distributing or establishing keys without leaking them to adversaries
10. A replay attack on a password-based protocol can be mitigated by:
- ☐ Storing the password in plaintext on the server
 - ☐ Using *nonce*-based challenges or timestamps to ensure each response is fresh
 - ☐ Disabling hashing in favor of direct key usage
 - ☐ Relying on IP address binding for user authentication
-

Part B

1. **Replays and Nonces**

- **Question:** Explain how a **nonce** can protect a protocol from replay attacks. Include a scenario where a replay could succeed if the nonce is not verified properly.

2. **Authenticated Key Establishment**

- **Question:** Compare a purely *unauthenticated* Diffie-Hellman exchange with an *authenticated* one (e.g., using digital signatures or shared secret-based proofs). What

security risks does unauthenticated DH pose, and how does authentication mitigate them?

3. **Forward Secrecy Concepts**

- **Question:** Define *forward secrecy* in your own words. Why is it desirable in protocols like TLS? Give an example of how failing to provide forward secrecy could harm a user after a key compromise.

4. **Reflection vs. Relay**

- **Question:** Differentiate **reflection** attacks from **relay** attacks in authentication protocols. Provide an example of a scenario or protocol each might exploit.

5. **Dictionary Attack Defenses**

- **Question:** In what ways can a system or protocol limit the success of offline dictionary attacks? Suggest both **technical** and **policy** solutions.

6. **DH-EKE**

- **Question:** Sketch a simplified Diffie-Hellman Encrypted Key Exchange (DH-EKE) message flow. Explain the significance of encrypting DH shares with the user's password.

7. **Single Sign-On Threats**

- **Question:** Single Sign-On (SSO) can increase convenience but also enlarge the attack surface if not designed carefully. Explain two significant security risks in an SSO system and how they can be mitigated.

Part C

1. **Number of Offline Guesses**

- **Question:** Suppose an attacker has intercepted a transcript from a password-based challenge-response protocol. They can test 10^7 password guesses per second offline. How many **hours** until they exhaust a dictionary of 10^{11} possible passwords?

2. **Diffie-Hellman Security**

- **Question:** A standard Diffie-Hellman exchange uses a **2048-bit** prime (p). If an attacker attempts to break it via discrete log, they can perform 2^{60} operations in a

feasible timeframe. Why is this still not enough for practical success, considering typical estimates for discrete log complexity (e.g., sub-exponential LMM or NFS algorithms)? *(Provide a short conceptual explanation rather than exact numeric estimates.)*

3. **Replay with Timestamps**

- **Question:** A protocol uses timestamps with a **2-minute** validity window to prevent replay. If an attacker obtains a valid message at time ($t=0$), how long can they replay it successfully (assuming no other checks)?

4. **Diffie-Hellman EKE Overhead**

- **Question:** In DH-EKE, each side encrypts the other's public DH share with a symmetric cipher keyed by the password. If encryption and decryption each take **~1ms**, and there are **2** exchanges, how many total milliseconds are spent purely on encryption/decryption overhead (ignoring exponentiations) per session?

5. **SSO: Single Sign-On Attempt Time**

- **Question:** In an SSO environment, each authentication to the SSO provider takes **~50ms**. A user accessing **5** federated services in quick succession triggers **5** SSO checks (one for each service). How many total milliseconds does the user wait on average due to these SSO calls (assuming no session caching)?
-